



Rapport de stage

Refonte et documentation de l'application CADI Web

CEA / INSTN
22 mai 2026

Stagiaire

Malek GHABI

Encadrant

TESTARD Vincent

Rapport de synthèse du travail réalisé autour de CADI et de sa documentation.

Table des matières

1	Introduction	3
2	Contexte du stage	4
2.1	Environnement	4
2.2	Application CADI	4
2.3	Problematique	4
3	Objectifs du travail	5
3.1	Objectif general	5
3.2	Objectifs detailles	5
4	Analyse de l'existant	6
4.1	Structure metier	6
4.2	Module EvalStat	6
4.3	Module IRIS	6
4.4	Module Bilans	6
4.5	Module Fiches de couts	6
5	Travail realise	7
5.1	Documentation fonctionnelle	7
5.2	Diagrammes UML	7
5.3	Reformulation des scenarios	7
5.4	Preparation de l'application web	7
6	Specifications des modules	8
6.1	EvalStat	8
6.2	IRIS	8
6.3	Bilans	8
6.4	Fiches de couts	8
6.5	Parametres et messagerie	8
7	Choix techniques	9
7.1	Python	9
7.2	FastAPI	9
7.3	React	9
7.4	PlantUML	9
7.5	LaTeX	9
8	Difficultes rencontrees	10
8.1	Complexite de l'existant	10
8.2	Fichiers et formats	10
8.3	Caracteres et encodage	10
8.4	Alignement fonctionnel	10

9 Bilan personnel	11
10 Perspectives	12
11 Conclusion	13

1. Introduction

Ce rapport presente le travail realise dans le cadre de la refonte et de la documentation de l'application CADI. Le projet concerne un outil interne utilise pour faciliter le suivi des formations, le traitement des evaluations, l'exploitation d'exports IRIS, la lecture des fiches de couts et la generation de bilans.

Le besoin principal etait de rendre plus clair un existant technique deja riche, puis de preparer une interface web permettant d'utiliser les traitements sans passer directement par des scripts Python ou des manipulations manuelles complexes.

2. Contexte du stage

2.1 Environnement

Le stage s'est deroule dans le contexte du CEA / INSTN. L'INSTN gere des activites de formation et doit produire plusieurs documents de suivi : evaluations stagiaires, bilans de sessions, bilans de formation, documents issus de fiches de couts et informations provenant d'exports IRIS.

Ces documents reposent sur des fichiers Excel, Word, CSV et sur des donnees provenant d'outils internes. La qualite du traitement depend donc fortement de la structure des fichiers, de la fiabilite des chemins reseau et de la coherence des donnees.

2.2 Application CADI

CADI est une application interne dont le coeur metier est developpe en Python. Le code existant contient une librairie `vte/` qui regroupe les principales classes et services :

- `Formation` et `Session` pour représenter les formations et les sessions ;
- `EvalStat` pour le traitement des evaluations ;
- `IRIS` pour la preparation des exports ;
- `BilanSessions` et `BilanFormation` pour la production de documents ;
- `FdC` pour la lecture des fiches de couts ;
- des classes techniques pour Excel, Word et la configuration.

2.3 Problematique

Le projet posait plusieurs questions :

- comment comprendre rapidement une base Python existante ;
- comment documenter les modules sans modifier leur logique metier ;
- comment transformer des traitements locaux en fonctionnalites web ;
- comment presenter les cas d'utilisation et les flux de maniere claire ;
- comment produire des documents PDF propres et exploitables.

3. Objectifs du travail

3.1 Objectif general

L'objectif general etait de preparer la modernisation de CADI en produisant une documentation claire et une vision fonctionnelle de l'application web cible. Il fallait identifier les modules existants, comprendre leurs responsabilites, formaliser les usages et decire les scenarios attendus.

3.2 Objectifs detailles

- analyser les fichiers Python du domaine metier ;
- identifier les classes, les attributs et les methodes importantes ;
- produire des diagrammes de classes, de sequence et de cas d'utilisation ;
- rediger un cahier des charges ;
- rediger des specifications fonctionnelles ;
- produire un rapport de stage coherent ;
- compiler les documents en PDF.

4. Analyse de l'existant

4.1 Structure metier

L'analyse du code a montre une separation entre le domaine, les services et l'infrastructure. Les classes du domaine portent les concepts principaux : formation, session, evaluation, bilan, fiche de couts et export IRIS. Les services orchestrent certains traitements, tandis que l'infrastructure gere les fichiers Excel, Word et les chemins de configuration.

4.2 Module EvalStat

Le module EvalStat traite les evaluations de stagiaires. Il s'appuie sur un trigramme de formation, un code IRIS et des fichiers d'evaluation. Le systeme distingue l'evaluation d'une session et l'evaluation globale de la formation. Une session peut donc etre traitee individuellement, puis ses resultats sont consolides dans le fichier de formation.

4.3 Module IRIS

Le module IRIS transforme des exports natifs en fichiers exploitables. Il gere plusieurs types d'exports : sessions, formations, ventes et inscriptions. Chaque type possede sa propre configuration, ce qui permet d'adapter les colonnes, les onglets, les modeles et les fichiers produits.

4.4 Module Bilans

Le module Bilans produit des documents de synthese. Le bilan de sessions s'interesse a une periode ou a une liste de codes IRIS. Le bilan de formation se concentre sur une annee et rassemble davantage d'informations : evaluations, ventes IRIS, fiche de couts et specifications.

4.5 Module Fiches de couts

Le module FdC lit une fiche de couts Excel. La logique du code repose sur un principe de facade et de lecteurs specialises. La classe principale masque la complexite et delegue la lecture a un lecteur compatible avec la version de la fiche.

5. Travail realise

5.1 Documentation fonctionnelle

Un cahier des charges et des specifications fonctionnelles ont ete rediges. Ces documents decrivent les besoins, les acteurs, les modules et les comportements attendus de l'application web. Un effort particulier a ete fait pour distinguer les actions utilisateur des traitements automatiques.

Par exemple, dans le module Fiches de couts, l'utilisateur selectionne seulement la fiche. La detection de version et le choix du lecteur sont automatiques. De meme, dans le module Bilans, le trigramme, l'annee ou la periode et le type de bilan sont places au meme niveau, car ce sont les informations de depart du traitement.

5.2 Diagrammes UML

Des diagrammes UML ont ete produits avec PlantUML. Ils couvrent :

- les cas d'utilisation generaux ;
- les cas d'utilisation par scenario ;
- les classes du domaine ;
- les classes d'infrastructure ;
- les sequences de traitement EvalStat, IRIS et Bilans ;
- l'architecture globale de l'application.

Ces diagrammes facilitent la lecture du projet pour une personne qui ne connait pas encore le code. Ils permettent aussi de verifier que les modules sont bien separees et que les responsabilites principales sont identifiees.

5.3 Reformulation des scenarios

Les scenarios ont ete reformules pour etre plus proches de l'usage reel. Le vocabulaire a ete simplifie et les actions inutiles ont ete retirees. Par exemple, l'action "telecharger le fichier traite" n'est plus consideree comme une action principale du module IRIS. Le systeme doit surtout indiquer que le fichier traite est disponible.

5.4 Preparation de l'application web

L'application cible repose sur une interface web et une API. L'interface doit permettre de saisir les informations, selectionner les fichiers et afficher les resultats. L'API doit controler les donnees, appeler le coeur metier Python et retourner un statut clair.

Cette organisation permet de conserver la logique existante tout en proposant une interface plus accessible. Le coeur metier n'est pas reecrit inutilement : il est reutilise et expose progressivement.

6. Specifications des modules

6.1 EvalStat

L'utilisateur doit pouvoir saisir un trigramme de formation, taper un code IRIS et sélectionner les fiches d'évaluation. Le système contrôle les données puis lance le traitement. Le résultat attendu est la mise à jour des données d'évaluation et l'affichage d'un statut clair.

6.2 IRIS

L'utilisateur sélectionne un export natif et choisit son type. Le traitement produit un fichier propre et exploitable. Les informations importantes sont le type d'export, le nombre de lignes traitées et l'emplacement du fichier résultat.

6.3 Bilans

Le module Bilans doit permettre de générer un bilan de sessions ou un bilan de formation. Les informations initiales sont saisies ensemble : trigramme, année ou période et type de bilan. Le système récupère ensuite les données nécessaires et produit un document Word.

6.4 Fiches de couts

La fiche de couts est sélectionnée par l'utilisateur. Le système détecte automatiquement la version du fichier et utilise le lecteur adapté. Les données extraites servent ensuite à alimenter les bilans ou à afficher les informations principales de la formation.

6.5 Parametres et messagerie

Le module Parametres regroupe la connexion, l'inscription, le mot de passe oublié, les profils utilisateurs, les droits administrateur et la messagerie. La messagerie est limitée à l'envoi d'un mail au chef d'unité. Un message par défaut est proposé et peut être modifié avant l'envoi.

7. Choix techniques

7.1 Python

Python reste le langage du coeur metier. Ce choix est logique car les traitements existants sont deja developpes dans ce langage et utilisent des librairies adaptees a la manipulation de fichiers.

7.2 FastAPI

FastAPI est adapte pour exposer les traitements sous forme de routes web. Il permet de recevoir les formulaires, les fichiers, les parametres et de retourner des reponses structurees a l'interface.

7.3 React

React permet de construire une interface utilisateur moderne avec des composants reutilisables. Chaque module peut etre represente par une page ou un composant specialise.

7.4 PlantUML

PlantUML a ete utilise pour produire les diagrammes. Son avantage est de permettre une documentation textuelle, versionnable et facile a corriger.

7.5 LaTeX

LaTeX a ete utilise pour produire les documents PDF. Il garantit une mise en page stable, une table des matieres propre et une presentation adaptee aux documents de stage.

8. Difficultes rencontrees

8.1 Complexite de l'existant

Le code existant contient plusieurs modules lies entre eux. Certaines classes ne sont pas seulement des structures de donnees : elles orchestrent aussi des traitements, ouvrent des fichiers et declenchent des generations de documents. Il a donc fallu analyser les relations avant de produire les diagrammes.

8.2 Fichiers et formats

CADI manipule plusieurs formats : Excel, Word, CSV, PDF et exports IRIS. Chaque format possede ses contraintes. Les fichiers peuvent aussi etre verrouilles, absents ou mal structures, ce qui oblige a prevoir des controles.

8.3 Caracteres et encodage

Des problemes d'affichage peuvent apparaitre dans les PDF lorsque l'encodage ou certaines polices ne sont pas bien prises en charge. Pour limiter ces erreurs, les documents ont ete reecrits avec une syntaxe simple et compilee proprement.

8.4 Alignement fonctionnel

Une autre difficulte a ete de distinguer ce que fait l'utilisateur et ce que fait automatiquement le systeme. Cette distinction est importante pour les diagrammes de cas d'utilisation : un cas d'utilisation ne doit pas transformer chaque operation interne en action utilisateur.

9. Bilan personnel

Ce travail m'a permis de progresser sur plusieurs aspects :

- lecture et comprehension d'un projet Python existant ;
- analyse de classes, methodes et relations ;
- redaction de documents techniques ;
- production de diagrammes UML ;
- structuration de specifications fonctionnelles ;
- compilation et correction de documents PDF.

Le projet m'a aussi montre l'importance de la documentation dans une application metier. Un code peut etre fonctionnel, mais difficile a reprendre si ses modules, ses entrees et ses sorties ne sont pas clairement expliques.

10. Perspectives

Plusieurs evolutions peuvent etre envisagees :

- finaliser l'interface React des modules principaux ;
- connecter chaque page a une route FastAPI ;
- ajouter une authentication complete ;
- journaliser les traitements ;
- securiser l'accès aux fichiers reseau ;
- ajouter des tests sur les traitements critiques ;
- enrichir les messages d'erreur et les retours utilisateur.

11. Conclusion

Le travail realise a permis de clarifier le fonctionnement de CADI et de preparer sa transformation en application web. Les documents produits structurent les besoins, les modules, les scenarios et les diagrammes. Ils constituent une base pour poursuivre le developpement avec une vision plus claire de l'existant et des objectifs.

La prochaine etape consiste a relier progressivement l'interface web aux traitements Python, en commençant par les modules les plus importants : EvalStat, IRIS, Bilans et Fiches de couts.